Roteiro do Trabalho Prático — Teste de Integração

Engenharia de Software III

Objetivo do trabalho prático

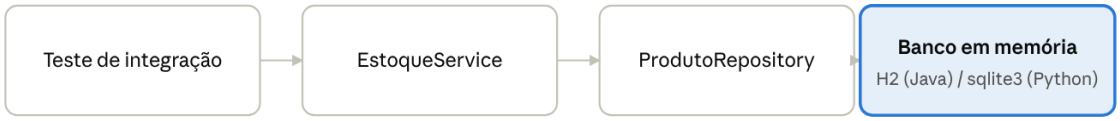
O trabalho prático aplica, na prática, os dois objetivos: compreender os níveis de teste de software e aplicar testes em nível de integração, verificando que o `EstoqueService` e o `ProdutoRepository` funcionam corretamente **juntos**, contra um banco de dados real (em memória).

- **Objetivo 1 — Níveis de teste:** o roteiro mostra na prática por que um teste de integração é diferente de um teste unitário — ele não isola o componente, ele verifica a integração entre peças reais.
- **Objetivo 2 — Aplicar testes de integração:** você vai escrever, de forma independente, os testes de integração que faltam no cenário demonstrado, seguindo o mesmo padrão.

Este roteiro parte do cenário apresentado (Sistema de Controle de Estoque) e assume que o exemplo guiado já foi acompanhado em sala.

O cenário: Sistema de Controle de Estoque

O teste atravessa o Service e o Repository até o banco real



Nada entre o Service e o banco é substituído por um dublê — só a infraestrutura troca para uma versão em memória.

fluxo · teste → EstoqueService → ProdutoRepository → banco em memória

É o mesmo cenário demonstrado: uma loja controla a quantidade de cada produto em estoque, usando um `EstoqueService` que se apoia em um `ProdutoRepository` para ler e gravar no banco de dados.

- **Produto** — id, nome e quantidade em estoque.
- **ProdutoRepository** — lê e grava produtos no banco (JDBC em Java, `sqlite3` em Python).
- **EstoqueService** — contém a regra de negócio: `adicionarEstoque` soma a quantidade recebida ao estoque; `venderProduto` subtrai a quantidade vendida, mas lança uma exceção — sem alterar o banco — quando a quantidade pedida é maior que o estoque disponível.

O trabalho prático usa exatamente essas três classes — o código completo está na seção "Código fornecido", abaixo.

Ferramentas e pré-requisitos

Você pode fazer o trabalho em Java, em Python, ou nos dois. Nenhuma ferramenta nova além das já usadas nos testes unitários — só adicionamos um banco de dados real para o teste de integração.

Item	Java	Python
Versão mínima	JDK 17+	Python 3.10+
Build / gerenciador	Maven (ou Gradle)	pip
Framework de teste	JUnit 5 (`junit-jupiter`)	pytest
Banco para o teste de integração	H2 Database em memória (`com.h2database:h2`)	`sqlite3` (já vem na biblioteca padrão)
Acesso ao banco	JDBC (`java.sql.*`)	módulo `sqlite3`
Comando para rodar os testes	`mvn test`	`pytest -v`

Nenhum servidor de banco de dados precisa ser instalado: tanto o H2 quanto o SQLite rodam em memória, dentro do próprio processo de teste.

Código fornecido (ponto de partida)

As três classes abaixo já estão prontas — **o trabalho de vocês é escrever os testes de integração, não implementar estas classes.** Elas representam o mesmo cenário demonstrado: um `EstoqueService` que usa um `ProdutoRepository` para ler e gravar produtos em um banco real.

Schema do banco (igual para as duas linguagens)

```
CREATE TABLE produtos (
    id INTEGER PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    quantidade INT NOT NULL
);
```

Java

```
// Produto.java
public class Produto {
    private Long id;
    private String nome;
    private int quantidade;

    public Produto(Long id, String nome, int quantidade) {
        this.id = id;
        this.nome = nome;
        this.quantidade = quantidade;
    }

    public Long getId() { return id; }
    public String getNome() { return nome; }
```

```
    public int getQuantidade() { return quantidade; }
}
```

```
// EstoqueInsuficienteException.java
public class EstoqueInsuficienteException extends RuntimeException {
    public EstoqueInsuficienteException(String mensagem) {
        super(mensagem);
    }
}
```

```
// ProdutoRepository.java
import java.sql.*;

public class ProdutoRepository {
    private final Connection conexao;

    public ProdutoRepository(Connection conexao) {
        this.conexao = conexao;
    }

    public void salvar(Produto produto) throws SQLException {
        String sql = "INSERT INTO produtos (id, nome, quantidade) VALUES (?, ?, ?)";
        try (PreparedStatement ps = conexao.prepareStatement(sql)) {
            ps.setLong(1, produto.getId());
            ps.setString(2, produto.getNome());
            ps.setInt(3, produto.getQuantidade());
            ps.executeUpdate();
        }
    }

    public Produto buscarPorId(Long id) throws SQLException {
        String sql = "SELECT * FROM produtos WHERE id = ?";
        try (PreparedStatement ps = conexao.prepareStatement(sql)) {
            ps.setLong(1, id);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                return new Produto(rs.getLong("id"), rs.getString("nome"),
rs.getInt("quantidade"));
            }
            return null;
        }
    }

    public void atualizarQuantidade(Long id, int novaQuantidade) throws SQLException
{
        String sql = "UPDATE produtos SET quantidade = ? WHERE id = ?";
        try (PreparedStatement ps = conexao.prepareStatement(sql)) {
            ps.setInt(1, novaQuantidade);
            ps.setLong(2, id);
            ps.executeUpdate();
        }
    }
}
```

```
// EstoqueService.java
import java.sql.SQLException;

public class EstoqueService {
    private final ProdutoRepository repositorio;

    public EstoqueService(ProdutoRepository repositorio) {
        this.repositorio = repositorio;
    }

    public void adicionarEstoque(Long produtoId, int quantidade) throws SQLException
{
        Produto produto = repositorio.buscarPorId(produtoId);
        int novaQuantidade = produto.getQuantidade() + quantidade;
        repositorio.atualizarQuantidade(produtoId, novaQuantidade);
    }

    public void venderProduto(Long produtoId, int quantidade) throws SQLException {
        Produto produto = repositorio.buscarPorId(produtoId);
        if (quantidade > produto.getQuantidade()) {
            throw new EstoqueInsuficienteException("Estoque insuficiente para o
produto " + produtoId);
        }
        repositorio.atualizarQuantidade(produtoId, produto.getQuantidade() -
quantidade);
    }
}
```

Python

```
# produto.py
class Produto:
    def __init__(self, id, nome, quantidade):
        self.id = id
        self.nome = nome
        self.quantidade = quantidade
```

```
# estoque_service.py (exceção)
class EstoqueInsuficienteError(Exception):
    pass
```

```
# produto_repository.py
class ProdutoRepository:
    def __init__(self, conexao):
        self.conexao = conexao

    def salvar(self, produto):
        self.conexao.execute(
            "INSERT INTO produtos (id, nome, quantidade) VALUES (?, ?, ?)",
            (produto.id, produto.nome, produto.quantidade),
        )
```

```
        self.conexao.commit()

    def buscar_por_id(self, produto_id):
        cursor = self.conexao.execute(
            "SELECT id, nome, quantidade FROM produtos WHERE id = ?", (produto_id,)
        )
        linha = cursor.fetchone()
        if linha is None:
            return None
        return Produto(*linha)

    def atualizar_quantidade(self, produto_id, nova_quantidade):
        self.conexao.execute(
            "UPDATE produtos SET quantidade = ? WHERE id = ?",
            (nova_quantidade, produto_id),
        )
        self.conexao.commit()
```

```
# estoque_service.py
class EstoqueService:
    def __init__(self, repositorio):
        self.repositorio = repositorio

    def adicionar_estoque(self, produto_id, quantidade):
        produto = self.repositorio.buscar_por_id(produto_id)
        nova_quantidade = produto.quantidade + quantidade
        self.repositorio.atualizar_quantidade(produto_id, nova_quantidade)

    def vender_produto(self, produto_id, quantidade):
        produto = self.repositorio.buscar_por_id(produto_id)
        if quantidade > produto.quantidade:
            raise EstoqueInsuficienteError(
                f"Estoque insuficiente para o produto {produto_id}"
            )
        self.repositorio.atualizar_quantidade(
            produto_id, produto.quantidade - quantidade
        )
```

Roteiro de execução passo a passo

Escolha Java, Python, ou os dois — a lógica é exatamente a mesma, só muda a sintaxe.

Para Java

1. Crie (ou abra) um projeto Maven com as dependências `junit-jupiter` e `com.h2database:h2` configuradas no `pom.xml`.
2. Copie as classes fornecidas (`Produto.java`, `EstoqueInsuficienteException.java`, `ProdutoRepository.java`, `EstoqueService.java`) para `src/main/java`.
3. Crie a classe de teste `EstoqueServiceIntegrationTest.java` em `src/test/java`.
4. No método `@BeforeEach`, abra uma conexão com um banco H2 em memória (`jdbc:h2:mem:estoque;DB_CLOSE_DELAY=-1`), execute o script de criação da tabela `produtos` e insira um produto inicial para os testes usarem.

5. Escreva `testAdicionarEstoqueAtualizaBanco()`: chame `adicionarEstoque`, depois consulte o banco **diretamente** (sem passar pelo service) para confirmar que a quantidade foi persistida.
6. Escreva `testVenderProdutoComEstoqueSuficiente()`: mesmo padrão, confirmando a baixa de estoque no banco.
7. Escreva `testVenderProdutoComEstoqueInsuficienteNaoAlteraBanco()`: tente vender mais do que o disponível, capture `EstoqueInsuficienteException` com `assertThrows`, e confirme que a quantidade no banco **não mudou**.
8. No `@AfterEach`, feche a conexão para isolar os testes uns dos outros.
9. Rode `mvn test` e confirme que todos os testes passam.
10. **Desafio extra:** escreva um teste que faz duas vendas seguidas do mesmo produto e confirma que o saldo final no banco é o esperado.

Para Python

1. Garanta que o `pytest` está instalado (`pip install pytest`) — `sqlite3` já vem na biblioteca padrão.
2. Copie os arquivos fornecidos (`produto.py`, `produto_repository.py`, `estoque_service.py`) para a pasta do projeto.
3. Crie o arquivo `test_estoque_service_integration.py`.
4. Crie uma fixture `conexao` que abra `sqlite3.connect(":memory:")`, execute o script de criação da tabela `produtos`, insira um produto inicial, dê `yield` da conexão e feche-a ao final (`conexao.close()`).
5. Escreva `test_adicionar_estoque_atualiza_banco`: chame `adicionar_estoque`, depois consulte o banco **diretamente** para confirmar a persistência.
6. Escreva `test_vender_produto_com_estoque_suficiente`: mesmo padrão, confirmando a baixa de estoque no banco.
7. Escreva `test_vender_produto_com_estoque_insuficiente_nao_altera_banco`: capture `EstoqueInsuficienteError` com `pytest.raises`, e confirme que o banco **não mudou**.
8. Rode `pytest -v` e confirme que todos os testes passam.
9. **Desafio extra:** escreva um teste parametrizado (`@pytest.mark.parametrize`) cobrindo diferentes quantidades inválidas de venda (zero, negativa, maior que o estoque).

Critérios de aceite

- ☐ Os três testes obrigatórios existem e rodam sem erro: `testAdicionarEstoqueAtualizaBanco`, `testVenderProdutoComEstoqueSuficiente` e `testVenderProdutoComEstoqueInsuficienteNaoAlteraBanco` (ou seus equivalentes em Python).
- ☐ Cada teste cria um banco em memória do zero antes de rodar (H2 ou sqlite `:memory:`) — nenhum teste usa um banco compartilhado ou um arquivo em disco.
- ☐ As asserções leem o estado de volta do banco de dados, nunca de variáveis mantidas em memória pelo objeto.
- ☐ O teste de estoque insuficiente confirma, via `assertThrows` / `pytest.raises`, que a exceção é lançada e que a quantidade no banco permanece inalterada.

- ☐ `mvn test` (ou `pytest -v`) mostra todos os testes passando, sem nenhum ignorado (`@Disabled` / `skip`).
- ☐ O desafio extra foi implementado e também passa.

Dicas e solução de problemas

- **"Table produtos already exists" ou erro ao recriar a tabela**: confirme que cada teste abre uma conexão nova e em memória. Se a conexão for reaproveitada entre testes, a tabela do teste anterior ainda existe.
- **Um teste passa isolado, mas falha ao rodar a suíte inteira**: sinal de dependência entre testes. Cada teste deve preparar seus próprios dados e fechar sua própria conexão ao final (`@AfterEach` ou o fechamento da fixture).
- **`NullPointerException` ou `AttributeError` ao chamar `buscarPorId`**: confirme que o produto foi inserido no banco antes do teste chamar o service — revise o `@BeforeEach` / a fixture `conexao`.
- **O teste de exceção não lança nada**: confira a condição de comparação (`quantidade > produto.getQuantidade()` / `quantidade > produto.quantidade`) e a ordem dos parâmetros passados para `venderProduto` / `vender_produto`.
- **Dúvida geral sobre a estrutura de um teste**: compare com o exemplo guiado do arquivo ppt (`testVenderProdutoComEstoqueSuficiente`) — os três testes pendentes seguem exatamente o mesmo padrão de arrange / act / assert.

Trabalho complementar: teste de integração em um projeto de outra disciplina

Para fixar o conceito além do cenário de estoque, cada dupla também deve escolher um projeto já desenvolvido em outra disciplina (pode ser de Engenharia de Software II, por exemplo) que tenha pelo menos duas camadas que se comunicam de verdade — uma regra de negócio e um acesso a dados, uma API e um banco, um controller e um service, etc.

- ☐ Identifiquem, no projeto escolhido, dois componentes que se comunicam de verdade (ex.: um Service que chama um Repository, ou um Controller que chama um Service).
- ☐ Escrevam pelo menos 2 testes de integração reais para essa comunicação, seguindo os mesmos princípios mostrados no arquivo ppt: dependência externa isolada (banco em memória quando possível), dados controlados e testes independentes entre si.
- ☐ Pelo menos um dos testes deve cobrir um caminho de erro/exceção — algo equivalente ao "estoque insuficiente" do exemplo de hoje.
- ☐ Apresentem o teste rodando (terminal ou IDE) e expliquem, em poucas palavras, por que aquilo é um teste de integração e não um teste unitário.

Esse trabalho complementar mostra que o conceito foi compreendido além do exemplo e deve ser apresentado junto com o trabalho do Sistema de Controle de Estoque.